Analysis of Algorithms
Lab 04

Felix R. Gonzalez

1 Introduction

The objective of this laboratory was to implement and analyze three different algorithms using C++:

- The 0/1 Knapsack problem using exhaustive search.
- Depth First Search (DFS) for graph traversal.
- Breadth First Search (BFS) for graph traversal.

The implementations were developed using C++ and tested by measuring their execution performance.

2 0/1 Knapsack Problem

2.1 Algorithm Description

The 0/1 Knapsack problem was solved using an exhaustive search approach.

The algorithm generates all possible combinations of items and evaluates each subset. A solution is accepted if the total weight does not exceed the capacity and its value is greater than the previous best solution.

The exhaustive search complexity is:

$$O(2^n)$$

because for n items there are 2^n possible combinations.

2.2 Experimental Results

The execution time was measured for different problem sizes.

The following table shows the obtained results:

n	Execution Time (ms)
3	0
4	0
5	0
6	0
7	0
8	0
9	0
10	0
11	0
12	2
13	4
14	6
15	8
16	17
17	27
18	48
19	74
20	174
21	310
22	625
23	1305
24	2610
25	6083
26	11048
27	22059
28	45166

Table 1: Execution times for exhaustive search Knapsack

2.3 Maximum Problem Size

The objective was to determine the maximum value of n that can be solved within 30 seconds.

From the experiments:

- $n = 27$ required 22059 ms (22 seconds).
- $n = 28$ required 45166 ms (45 seconds).

Therefore, the largest problem size solved under 30 seconds was:

$$\boxed{n = 27}$$

2.4 Example Solution

For a problem size of $n = 5$, the generated items were:

Item	Weight	Value
0	9	17
1	3	10
2	5	12
3	6	5
4	9	19

Table 2: Generated Knapsack instance

The selected items were:

$$\{0, 2, 4\}$$

with:

$$Total\ Weight = 23$$

and:

$$Maximum\ Value = 48$$

3 Depth First Search (DFS)

3.1 Algorithm Description

Depth First Search explores a graph by visiting a node and then recursively visiting its unvisited neighbors before returning to previous nodes.

The graph was represented using an adjacency list because it is efficient for sparse graphs.

The complexity of DFS is:

$$O(V + E)$$

where V represents the number of vertices and E represents the number of edges.

3.2 Results

The DFS traversal starting from node 0 was:

$$0 \to 1 \to 3 \to 4 \to 5 \to 2$$

The execution result was:

```
DFS traversal:
0 1 3 4 5 2
```

4 Breadth First Search (BFS)

4.1 Algorithm Description

Breadth First Search visits nodes level by level using a queue data structure.

The graph representation was also an adjacency list.

The complexity of BFS is:

$$O(V + E)$$

4.2 Results

The BFS traversal starting from node 0 was:

$$0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$$

The execution result was:

```
BFS traversal:
0 1 2 3 4 5
```

5 Execution Screenshots

The execution screenshots of the program were included as evidence of the obtained results.

Insert your screenshot here

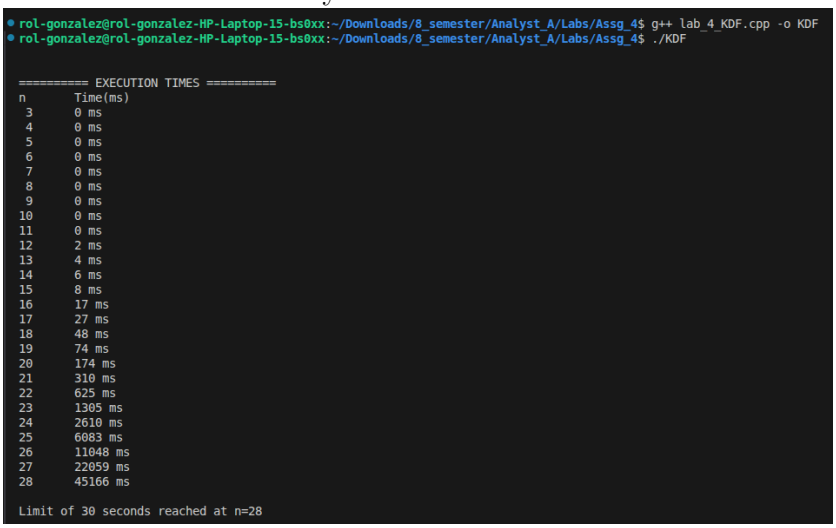


Figure 1: Program execution

Insert your screenshot here

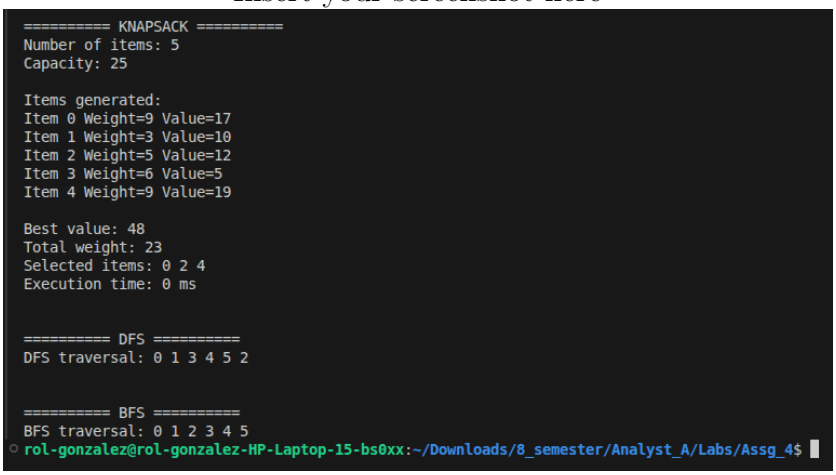


Figure 2: Program execution

6 Conclusion

The exhaustive search approach guarantees the optimal solution for the Knapsack problem, but its execution time increases exponentially as the number of items grows.

The experiments showed that the maximum problem size solved under 30 seconds was $n = 27$.

DFS and BFS successfully traversed the graph using adjacency list representation. DFS explores the graph deeply using recursion, while BFS explores nodes level by level using a queue.